

1. Plan de test unitaire

1.1. **Identification du test**

TestUniaire_ValidationPointMesures_v1

1.2. **Référence au module testé**

Méthode testé : bool ValidationPointMesure(List<PointLatLng>
point_mesures)

Cette méthode appartient à la classe lhm et aura comme valeur de retour

Valeur de retour	Description
is_valide	true si les points de mesure sont valide

1.3. **Objectif du test**

Ce test aura pour objectif de tester la bonne vérification des points de mesure par la méthode ValidationPointMesure en lui insérant des points de mesure factice

1.4. **Procédure de test**

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point avec la coordonnées spécifiée dans les valeurs de test (une liste par cas de test)
- Passage de la variable dans la méthode testé
- Lancement du projet
- Comparaison des valeurs de retour de la méthode testé avec les résultat attendu (voir partie 1.5)

1.4.1. Valeurs de test

Nom du test	Description du vecteur test
TC_01	points_mesures = {(47.567, -0.5145)}
TC_02	points_mesures = {(90, 180)}
TC_03	points_mesures = {(-90; -180)}
TC_04	points_mesures = ((zone[0].Point[0].Lat + zone[0].Point[2].lat)/2; (zone[0].Point[0].Lng+zone[0].Point[2].Lng)/2)
TC_05	points_mesures = {null}
TC_06	points_mesures = {(1000.890; 2000.345)}
TC_07	points_mesures = {(47.567, -0.5145) (48.567, -0.6145)}
TC_08	points_mesures = {(47.567, -0.5145) (67.567, -50.5145)}
TC_09	points_mesures = {null, (47.567, -0.5145)}
TC_10	points_mesures = {(1000.878; 2000.543), (47.567, -0.5145)}
TC_11	points_mesures = {(47.567, -0.5145) (47.567, -0.5145)}
TC_12	points_mesures = ((zone[0].Point[0].Lat - zone[0].Point[2].lat)/2; (zone[0].Point[0].Lng - zone[0].Point[2].Lng)/2)

1.4.2. TC_01 : Cas valide

a. Initialisation :

points_mesures est une **liste de PointLatLng contenant un point de coordonnées (47.567;-0.5145)**

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point de coordonnées (47.567, -0.5145)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode et la valeur attendu grâce à la méthode **Assert.IsTrue()**

la méthode est valide si le Assert.IsTrue **ne retourne pas d'exception**

1.4.3. TC_02 : Cas limite positif

a. Initialisation :

points_mesures est une liste de PointLatLng contenant un point de coordonnées (90 ; 180)

b. Lancement :

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point de coordonnées (90, 180)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat :

Pour observer le résultat on compare la valeur retournée par la méthode et la valeur attendu grâce à la méthode **Assert.IsTrue()**

la méthode est valide si le Assert.IsTrue **ne retourne pas d'exception**

1.4.4. TC_03 : Cas limite négatif

a. Initialisation :

points_mesure est une liste de PointLatLng contenant un point de coordonnées (-90 ; -180)

b. Lancement :

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point de coordonnées (-90, -180)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat :

Pour observer le résultat on compare la valeur retournée par la méthode et la valeur attendu grâce à la méthode **Assert.IsTrue()**

la méthode est valide si le Assert.IsTrue **ne retourne pas d'exception**

1.4.5. TC_04 : Cas valide (point dans la zone)

a. Initialisation :

points_mesure est une **liste de PointLatLng contenant un point de coordonnées :**

$$\left(\frac{zone[0].Point[0].Lat + zone[0].Point[2].Lat}{2} ; \frac{zone[0].Point[0].Lng + zone[0].Point[2].Lng}{2} \right)$$

b. Lancement :

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant le point cité ci-dessus
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat :

Pour observer le résultat on compare la valeur retournée par la méthode et la valeur attendu grâce à la méthode **Assert.IsTrue()**

la méthode est valide si le Assert.IsTrue **ne retourne pas d'exception**

1.4.6. TC_05 : Cas valeur null

a. Initialisation :

points_mesures est une liste de PointLatLng contenant un point null

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point null
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat

Pour observer le résultat on compare l'exception retourner par Assert.ThrowsException<> et l'exception que doit retourner la méthode

la méthode est valide si le Assert.ThrowsException<> retourne une exception de type ArgumentNullException

1.4.7. TC_06 : Cas invalide

a. Initialisation :

points_mesure est une liste de PointLatLng contenant un point de coordonnées (1000.890; 2000.345)

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point de coordonnées (1000.890; 2000.345)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat

Pour observer le résultat on compare l'exception retourner par **Assert.ThrowsException<>** et l'exception que doit retourner la méthode

la méthode est valide si le Assert.ThrowsException<> retourne une exception de type **ArgumentException**

1.4.8. TC_07 : Cas valide (deux point proche)

a. Initialisation :

points_mesures est une **liste de PointLatLng contenant deux point de coordonnées (47.567; -0.5145) et de coordonnées (48.567, -0.6145)**

b. Lancement :

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant deux point de coordonnées (47.567; -0.5145) et de coordonnées (48.567, -0.6145)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat :

Pour observer le résultat on compare la valeur retournée par la méthode et la valeur attendu grâce à la méthode **Assert.IsTrue()**

la méthode est valide si le Assert.IsTrue **ne retourne pas d'exception**

1.4.9. TC_08 : Cas valide (deux point éloigné)

a. Initialisation :

points_mesures est une **liste de PointLatLng contenant deux point de coordonnées (47.567; -0.5145) et de coordonnées (67.567, -50.5145)**

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant deux point de coordonnées (47.567; -0.5145) et de coordonnées (67.567, -50.5145)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat

Pour observer le résultat on compare la valeur retournée par la méthode et la valeur attendu grâce à la méthode **Assert.IsTrue()**

la méthode est valide si le Assert.IsTrue **ne retourne pas d'exception**

1.4.10. TC_09 : Cas une valeur null (une valeur null, et une valeur valide)

a. Initialisation :

points_mesures est une **liste de PointLatLng contenant un point null et un point de coordonnées (47.567; -0.5145)**

b. Lancement :

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point null et un point de coordonnées (47.567; -0.5145)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat :

Pour observer le résultat on compare l'exception retourner par **Assert.ThrowsException<>** et l'exception que doit retourner la méthode

la méthode est valide si le Assert.ThrowsException<> retourne une exception de type **ArgumentNullException**

1.4.11. TC_10 : Cas invalide (un point invalide et un autre valide)

a. Initialisation :

points_mesures est une liste de PointLatLng contenant un point de coordonnées (1000.890; 2000.345) et un point de coordonnées(47.567; -0.5145)

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant un point de coordonnées (1000.890; 2000.345)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat

Pour observer le résultat on compare l'exception retourner par Assert.ThrowsException<> et l'exception que doit retourner la méthode

la méthode est valide si le Assert.ThrowsException<> retourne une exception de type ArgumentException

1.4.12. TC_11 : Cas invalide (deux point identique)

a. Initialisation :

points_mesures est une liste de PointLatLng contenant deux point de coordonnées (47.567; -0.5145) et de coordonnées (47.567; -0.5145)

b. Lancement :

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant deux point de coordonnées (47.567; -0.5145) et de coordonnées (47.567; -0.5145)
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat :

Pour observer le résultat on compare l'exception retourner par **Assert.ThrowException<>** et l'exception que doit retourner la méthode

la méthode est valide si le Assert.ThrowException<> retourne une exception de type **ArgumentException**

1.4.13. TC_12 : Cas invalide (point pas dans la zone)

a. Initialisation :

points_mesures est une **liste de PointLatLng contenant un point de coordonnée:**

$$\left(\frac{zone[0].Point[0].Lat - zone[0].Point[2].Lat}{2} ; \frac{zone[0].Point[0].Lng - zone[0].Point[2].Lng}{2} \right)$$

b. Lancement :

- Ouverture du projet de test
- Configuration d'une liste de PointLatLng contenant le point cité ci-dessus
- Passage de la variable dans la méthode testé
- Lancement du projet

c. Observation des résultat :

Pour observer le résultat on compare l'exception retourner par **Assert.ThrowException<>** et l'exception que doit retourner la méthode

la méthode est valide si le Assert.ThrowException<> retourne une exception de type **ArgumentException**

1.5. *Résultat attendu*

Nom du test	Résultat attendu
TC_01	La méthode renvoie true
TC_02	La méthode renvoie true
TC_03	La méthode renvoie true
TC_04	la méthode renvoie true
TC_05	La méthode renvoie une exception de type ArgumentNullException
TC_06	La méthode renvoie une exception de type ArgumentException disant que les dimensions d'un point dans la liste sont erroné
TC_07	La méthode renvoie true
TC_08	La méthode renvoie true
TC_09	La méthode renvoie une exception de type ArgumentNullException
TC_10	La méthode renvoie une exception de type ArgumentException disant que les dimension d'un point dans la liste sont erroné
TC_11	La méthode renvoie une exception de type ArgumentException disant que deux point ne peuvent pas avoir la même coordonnées
TC_12	La méthode renvoie une exception de type ArgumentException disant que le point de mesure n'est pas dans la zones lié au point de mesure

1.6. *Moyen à mettre en oeuvre*

Moyen pour mettre en oeuvre les tests :

- PC portable/fixe avec Windows 10/11 d'installé dessus
- Visual studio configurer pour pouvoir exécuter des projets C# et des projets MS Test
- Un écran (si pc fixe)

2. Procès Verbal

Élément testé	Ihm::ValidationPointMesure			
Objectif du test	Ce test aura pour objectif de tester la bonne vérification des points de mesure par la méthode ValidationPointMesure en lui insérant des points de mesure factice			
Nom du testeur	BELAIRE Kévin		Date	07/04/2026
Moyen mis en oeuvre	Logiciel : Visual Studio 2022	Matériel : PC + Projet de test		Outil de développement : Framework .NET, Framework MS_Test
Procédure du test : - Ouverture du projet de test - Configuration d'une liste de PointLatLng contenant un point avec la coordonnées spécifiée dans les valeurs de test (une liste par cas de test) - Passage de la variable dans la méthode testé - Lancement du projet - Comparaison des valeurs de retour de la méthode testé avec les résultat attendu (voir partie 1.5)				
Id	Description du vecteur test	Résultat attendu	Résultat obtenu	Validation (O/N)
TC_01	points_mesures = {{(47.567, -0.5145)}}	La méthode renvoie true		
TC_02	points_mesures = {{(90, 180)}}	La méthode renvoie true		
TC_03	points_mesures = {{(-90; -180)}}	La méthode renvoie true		
TC_04	points_mesures = ((zone[0].Point[0].Lat + zone[0].Point[2].Lat)/2; (zone[0].Point[0].Lng+zone[0].Point[2].Lng)/2)	la méthode renvoie true		
TC_05	points_mesures = {null}	La méthode renvoie une exception de type ArgumentNullException		
TC_06	points_mesures = {{(1000.890; 2000.345)}}	La méthode renvoie une exception de type ArgumentException disant que les dimensions d'un point dans la liste sont erroné		
TC_07	points_mesures = {{(47.567, -0.5145) (48.567, -0.6145)}}	La méthode renvoie true		
TC_08	points_mesures = {{(47.567, -0.5145) (67.567, -50.5145)}}	La méthode renvoie true		
TC_09	points_mesures = {null, (47.567, -0.5145)}	La méthode renvoie une exception de type ArgumentNullException		
TC_10	points_mesures = {{(1000.878; 2000.543),	La méthode renvoie une exception de type ArgumentException disant que		

	(47.567, -0.5145)}	les dimension d'un point dans la liste sont erroné		
TC_11	points_mesures = {(47.567, -0.5145) (47.567, -0.5145)}	La méthode renvoie une exception de type ArgumentException disant que deux point ne peuvent pas avoir la même coordonnées		
TC_12	points_mesures = ((zone[0].Point[0].Lat - zone[0].Point[2].Lat)/2; (zone[0].Point[0].Lng - zone[0].Point[2].Lng)/2)	La méthode renvoie une exception de type ArgumentException disant que le point de mesure n'est pas dans la zones lié au point de mesure		